

Pipelined RISC-V Processor Design

Team 37

Jai Srikar M (2024102041)

Mohammad Shabaj (2024102016)

Nelagonda Pavaneshwar (2024102063)

Contents

1	Introduction	2
2	Project Overview	3
2.1	Architecture Summary	3
2.2	Supported Instructions	3
2.3	Pipeline Registers	3
3	Hazard Handling Mechanisms	4
3.1	Data Hazards and Forwarding	4
3.1.1	The Problem: Read-After-Write Hazards	4
3.1.2	The Solution: Data Forwarding Unit	4
3.2	Load-Use Hazards and Stalling	7
3.2.1	The Problem	7
3.2.2	The Solution: Hazard Detection Unit	7
3.3	Control Hazards and Branch Prediction	9
3.3.1	The Problem	9
3.3.2	The Solution: 2-Bit Dynamic Branch Predictor	9
4	Testing and Verification	10
4.1	Simulation Commands	10
4.2	Test Program	10
4.3	Waveform Analysis (GTKWave)	10
4.4	Final Register State	11
5	Challenges Faced	12
6	Team Contributions	12
7	Conclusion	12

1 Introduction

This project implements a 64-bit, 5-stage pipelined RV64I processor in Verilog, building upon a previously verified single-cycle design. Pipelining overlaps the execution of consecutive instructions to increase throughput.

The five pipeline stages are:

- **IF** — Instruction Fetch
- **ID** — Instruction Decode / Register Read
- **EX** — Execute (ALU)
- **MEM** — Memory Access
- **WB** — Write Back

Pipelining introduces structural hazards that must be resolved to maintain correctness. The design integrates robust hazard handling mechanisms: a Data Forwarding Unit for RAW hazards, a Hazard Detection Unit for load-use stalls, and a 2-bit dynamic branch predictor with hardware flushing for control hazards.

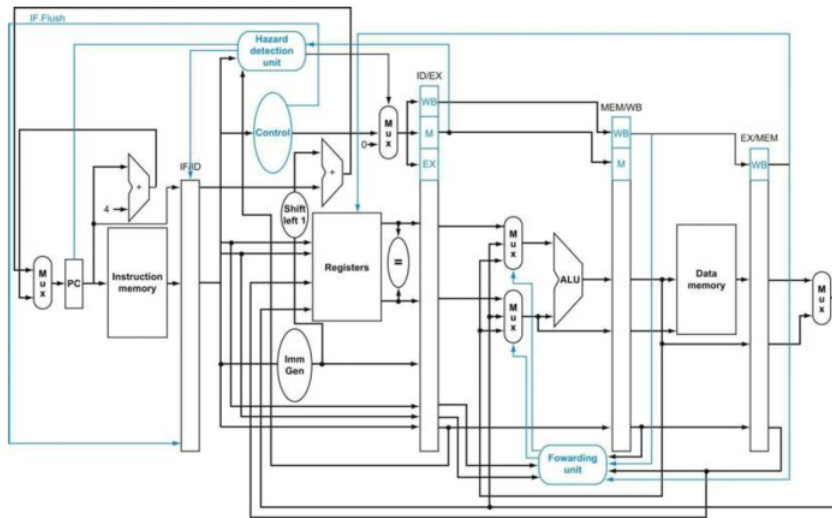


FIGURE 4.62 The final datapath and control for this chapter.

Figure 1: Overall architecture of the 5-stage pipelined processor with forwarding paths.

2 Project Overview

2.1 Architecture Summary

- 5-stage pipelined datapath
- 64-bit data and address width
- Byte-addressable memory
- Data Forwarding Unit (resolves EX and MEM hazards)
- Hazard Detection Unit (detects and stalls on load-use hazards)
- Dynamic branch predictor — 2-bit saturating counter with BHT and BTB

2.2 Supported Instructions

Type	Instructions
R-type	add, sub, and, or
I-type	addi
Load	ld
Store	sd
Branch	beq

2.3 Pipeline Registers

Pipeline registers act as stage boundaries, storing the intermediate values needed by each successive stage:

- IF/ID — holds the current PC and fetched instruction. Supports *write-enable* (frozen during load-use stalls) and *flush* (cleared on branch mispredictions).
- ID/EX — holds register read data, sign-extended immediate, **rs1/rs2/rd** pointers, and all control signals. Supports *bubble* insertion (load-use stall) and *flush* (misprediction).
- EX/MEM — holds the ALU result, computed branch target, forwarded store data (**rs2**), Zero flag, and propagated control signals.
- MEM/WB — holds memory read data, ALU result, destination register **rd**, and write-back control signals.

3 Hazard Handling Mechanisms

3.1 Data Hazards and Forwarding

3.1.1 The Problem: Read-After-Write Hazards

In a 5-stage pipeline an ALU result is not written to the register file until the WB stage (2 cycles after EX). Instructions that follow and read the same register will therefore fetch stale values during their own ID stage. Two variant distances arise:

- **EX Hazard** — the instruction *immediately* following the producer needs the result in its EX stage, one cycle before the producer reaches WB.
- **MEM Hazard** — the instruction *two* positions after the producer needs the result in its EX stage, still one cycle before the producer writes back.

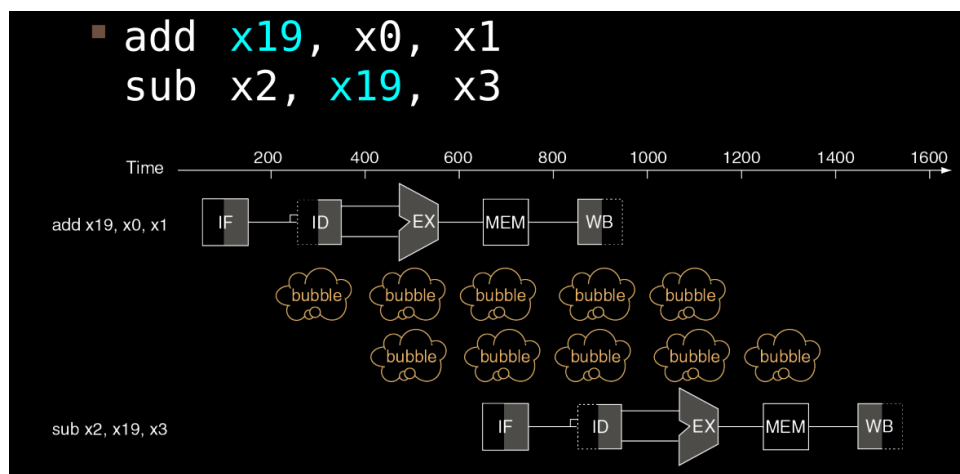


Figure 2: Illustration of 2-cycle stalls caused by RAW hazards without forwarding.

3.1.2 The Solution: Data Forwarding Unit

The Forwarding Unit continuously compares the destination registers in the EX/MEM and MEM/WB stages against the source registers in the EX stage and, when a hazard is detected, routes the fresh value directly to the ALU input multiplexers instead of reading from the register file.

- **EX Hazard** — forward from EX/MEM pipeline register (`ForwardA/B = 10`).
- **MEM Hazard** — forward from MEM/WB pipeline register (`ForwardA/B = 01`).
- **Priority** — if both hazards fire on the same operand simultaneously, the EX hazard takes priority (more recent value).
- **Store Forwarding** — a dedicated `store_data` mux applies the same forwarding logic to the `rs2` operand of `sd` instructions, ensuring the correct value is written to data memory even under a data hazard.

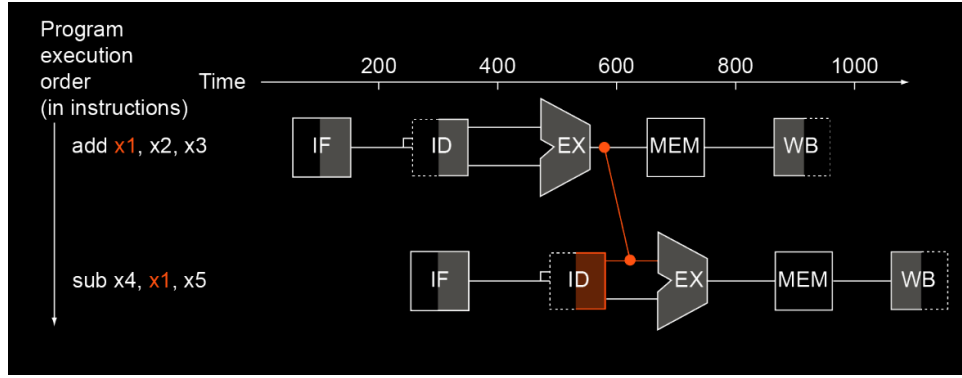


Figure 3: Forwarding unit resolving an EX hazard from EX/MEM directly to ALU inputs.

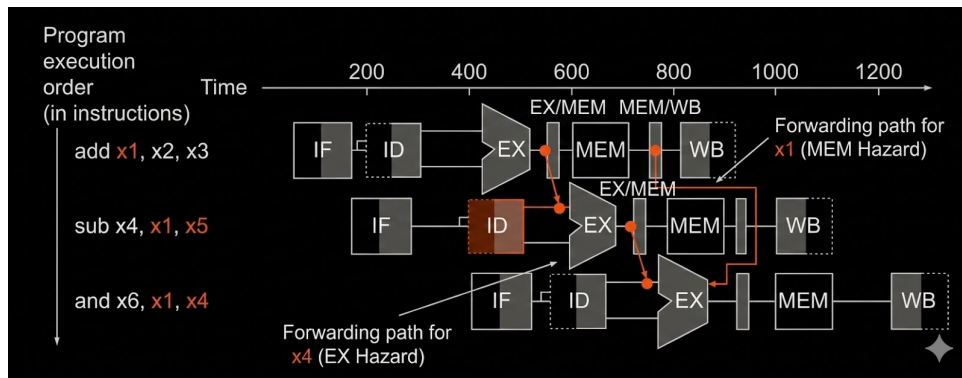


Figure 4: Forwarding unit resolving a MEM hazard from MEM/WB directly to ALU inputs.

The forwarding logic implemented in `forwarding_unit.v`:

```
// EX Hazard
if (EX_MEM_RegWrite && (EX_MEM_rd != 5'd0) && (EX_MEM_rd == ID_EX_rs1))
    ForwardA = 2'b10;
if (EX_MEM_RegWrite && (EX_MEM_rd != 5'd0) && (EX_MEM_rd == ID_EX_rs2))
    ForwardB = 2'b10;

// MEM Hazard (lower priority -- only when no EX hazard on same operand)
if (MEM_WB_RegWrite && (MEM_WB_rd != 5'd0) &&
    !(EX_MEM_RegWrite && (EX_MEM_rd != 5'd0) && (EX_MEM_rd == ID_EX_rs1)) &&
    (MEM_WB_rd == ID_EX_rs1))
    ForwardA = 2'b01;

if (MEM_WB_RegWrite && (MEM_WB_rd != 5'd0) &&
    !(EX_MEM_RegWrite && (EX_MEM_rd != 5'd0) && (EX_MEM_rd == ID_EX_rs2)) &&
    (MEM_WB_rd == ID_EX_rs2))
    ForwardB = 2'b01;
```

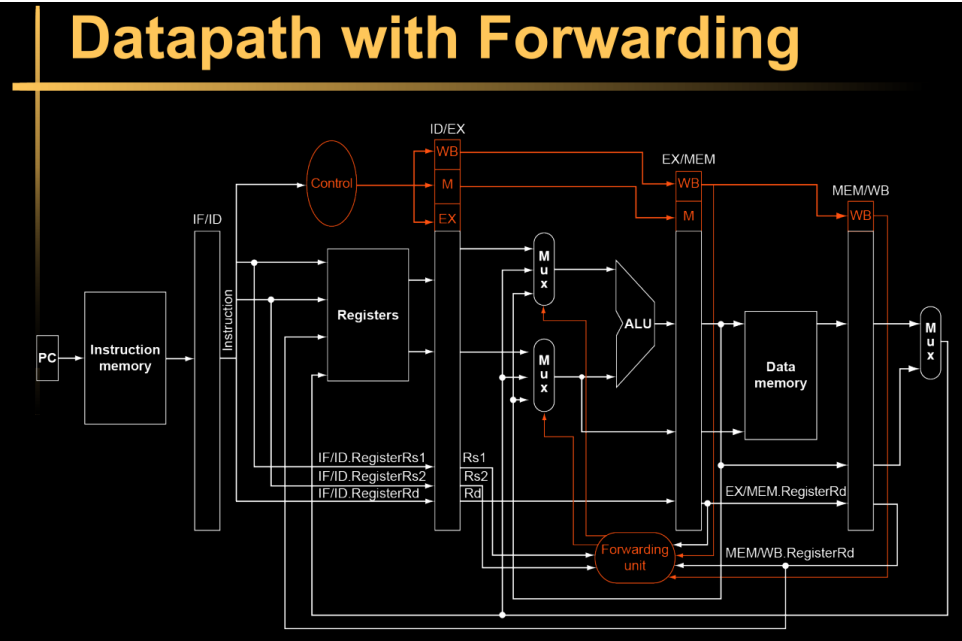


Figure 5: Complete datapath with Forwarding Unit.

3.2 Load-Use Hazards and Stalling

3.2.1 The Problem

ld instructions retrieve their result from data memory during the MEM stage. If the immediately following instruction needs this value in its EX stage, both events occur in the *same* clock cycle — forwarding backwards in time is physically impossible.

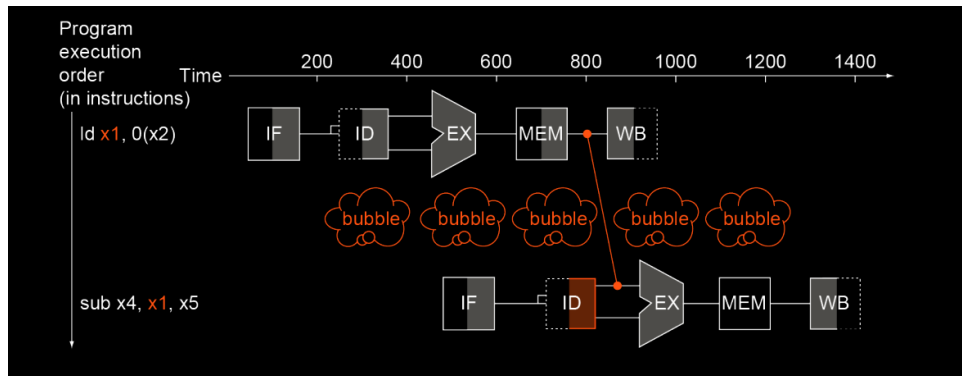


Figure 6: Load-Use hazard: the dependent instruction must wait 1 cycle before forwarding is possible.

3.2.2 The Solution: Hazard Detection Unit

The Hazard Detection Unit (`hazard_detection.v`) inserts a single NOP bubble to delay the dependent instruction by one cycle. After the stall, the loaded value becomes available in the MEM/WB register and can be forwarded normally.

The stall is achieved by:

1. Asserting `PC_write = 0` — freezes the PC (no new fetch).
2. Asserting `IF_ID_write = 0` — freezes the IF/ID register.
3. Asserting `ID_EX_bubble = 1` — clears all control signals in ID/EX (NOP).

Trigger condition (from `hazard_detection.v`):

```
if (ID_EX_MemRead &&
    ((ID_EX_rd == IF_ID_rs1) || (ID_EX_rd == IF_ID_rs2))) begin
    PC_write = 1'b0;
    IF_ID_write = 1'b0;
    ID_EX_bubble = 1'b1;
end
```

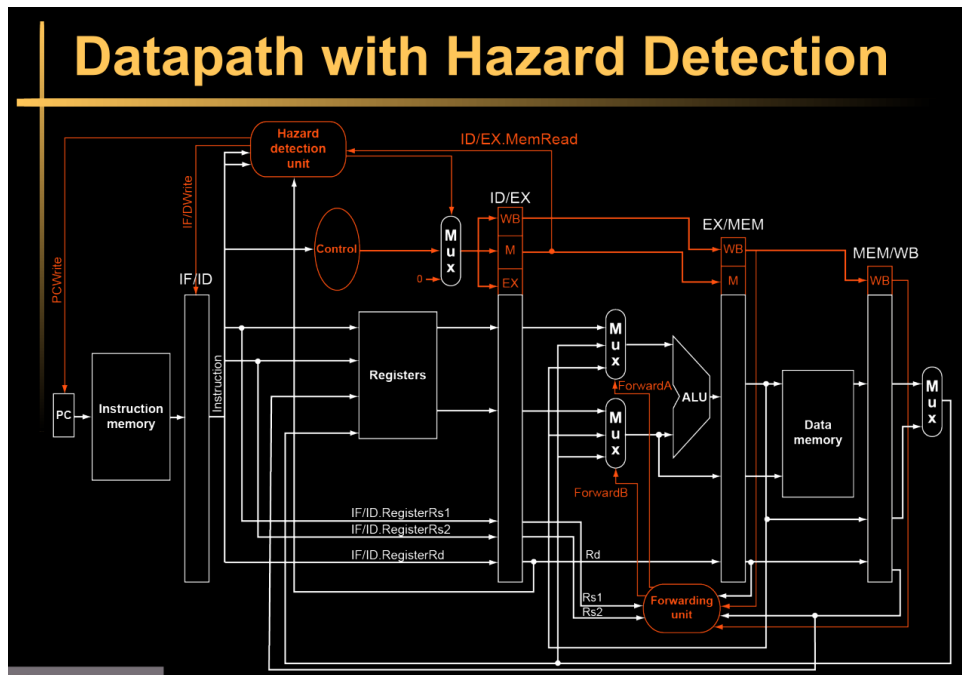



Figure 7: Complete datapath with Hazard Detection Unit.

3.3 Control Hazards and Branch Prediction

3.3.1 The Problem

When a branch instruction (`beq`) is fetched, the processor cannot know the correct next PC until the branch condition is evaluated in the EX stage — two cycles later. Blindly continuing to fetch sequential instructions risks loading wrong code into the pipeline.

3.3.2 The Solution: 2-Bit Dynamic Branch Predictor

The processor uses a 64-entry Branch History Table (BHT) combined with a Branch Target Buffer (BTB), both indexed by `PC[7:2]`, implemented in `branch_predictor.v`.

- **BHT** — one 2-bit saturating counter per entry: 00 = Strongly Not Taken, 01 = Weakly Not Taken, 10 = Weakly Taken, 11 = Strongly Taken. All entries are initialised to 01 (Weakly Not Taken) on reset.
- **BTB** — stores the most recently seen branch target for each entry.

The predictor operates as follows:

- **Prediction (IF stage)** — `BHT[index][1]` is the prediction bit. If 1 (taken), the PC jumps to `BTB[index]`; otherwise `PC+4` is fetched.
- **Resolution (EX stage)** — the actual branch outcome is evaluated.
- **Counter update** — the BHT counter is incremented (taken) or decremented (not taken), saturating at 11 and 00.
- **Misprediction flush (2-cycle penalty)** — if the prediction was wrong, both IF/ID and ID/EX are flushed and the PC is corrected.

PC selection logic:

```
assign pc_next = mispredict ? correct_pc :  
               predict_taken ? predicted_pc :  
               pc_plus4;
```

4 Testing and Verification

4.1 Simulation Commands

```
iverilog pipe_tb.v  
./a.out
```

4.2 Test Program

The regression test in `instructions_exp.txt` exercises all instruction types and every hazard mechanism:

```
addi x2, x0, 5 # x2 = 5  
addi x3, x0, 10 # x3 = 10  
add x1, x2, x3 # x1 = 15 (EX hazard: x2, x3)  
sub x2, x2, x3 # x2 = -5 (EX hazard: x2, x3)  
and x4, x3, x3 # x4 = 10  
and x5, x3, x4 # x5 = 10 (EX hazard: x4)  
or x6, x2, x4 # x6 = -5 (MEM hazard: x2)  
or x7, x2, x3 # x7 = -5  
sd x1, 0(x5) # mem[10] = 15  
ld x10, 0(x5) # x10 = 15 (load-use stall on next use)  
sd x6, 24(x5) # mem[34] = -5  
ld x11, 24(x5) # x11 = -5 (load-use stall on next use)  
beq x4, x5, 8 # taken -> skip infinite loop  
beq x0, x0, 0 # infinite loop trap (skipped)  
add x13, x1, x10 # x13 = 30 (MEM hazard: x10)  
add x0, x0, x0 # pipeline drain NOPs  
add x0, x0, x0  
add x0, x0, x0  
add x0, x0, x0
```

4.3 Waveform Analysis (GTKWave)

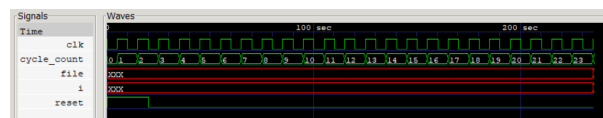


Figure 8: Snapshot of gtkwave

4.4 Final Register State

After simulation the testbench writes all 32 registers to `register_file.txt`. The expected values for key registers are:

Reg	Hex value	Dec	Note
x1	000000000000000f	15	add x1, x2, x3
x2	fffffffffffffffffb	-5	sub x2, x2, x3
x3	000000000000000a	10	addi x3, x0, 10
x4	000000000000000a	10	and x4, x3, x3
x5	000000000000000a	10	and x5, x3, x4
x6	fffffffffffffffffb	-5	or x6, x2, x4
x7	fffffffffffffffffb	-5	or x7, x2, x3
x10	000000000000000f	15	loaded from mem[10]
x11	fffffffffffffffffb	-5	loaded from mem[34]
x13	000000000000001e	30	add x13, x1, x10

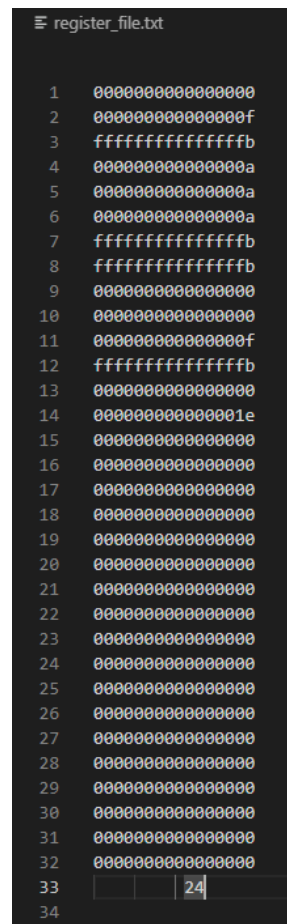


Figure 9: Snapshot of `register_file.txt` matching the expected values above.

5 Challenges Faced

- Correctly propagating `RegWrite` and `MemRead` control signals through all four pipeline registers without corruption.
- Resolving simultaneous EX and MEM hazards on the same operand via strict priority control in the Forwarding Unit.
- Ensuring that flushed NOP instructions did not inadvertently assert `RegWrite` or `MemWrite`, which would corrupt architectural state.
- Forwarding the correct `rs2` value for `sd` instructions (`store_data` mux) so that the value written to memory also reflects any active forwarding.
- Coordinating the load-use stall and branch-prediction flush so that both mechanisms interact correctly when they fire close together.

6 Team Contributions

- **Jai Srikar M (2024102041):** Implemented the 5-stage pipeline datapath including all four pipeline registers (IF/ID, ID/EX, EX/MEM, MEM/WB), the instruction fetch stage,
- **Mohammad Shabaj (2024102016):** Designed and integrated the Data Forwarding Unit to resolve EX and MEM data hazards, including forwarding for `sd` store data, ensuring the register file is bypassed correctly and helped with the report
- **Nelagonda Pavanewar (2024102063):** Implemented the Hazard Detection Unit for load-use stalls and the Flush Unit (`flush_unit.v`) and the 2-bit dynamic branch predictor (BHT + BTB) for control hazard handling for misprediction recovery and also helped with report

7 Conclusion

We successfully upgraded a single-cycle RV64I design into a fully pipelined 5-stage processor. The implementation handles all three classes of pipeline hazards: RAW data hazards are eliminated without stalls via the Forwarding Unit; load-use hazards are resolved with a single-cycle stall by the Hazard Detection Unit; and control hazards are minimised by a 2-bit saturating counter branch predictor backed by a BHT and BTB. Simulation confirmed correct register-file state for the complete test program.